

# Factory\_FW

---

## Sanitized As-Built Reference

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>System</b>           | Factory_FW, governed agentic design and delivery framework                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Document</b>         | Sanitized as-built reference                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Version</b>          | 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Status</b>           | v1 — Public Edition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Date</b>             | 17 August 2026                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Author</b>           | Martin Ždimera                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Audience</b>         | Solution architects, engineering leaders, platform teams, and technical reviewers                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Context and Role</b> | <p>Scope: the reusable framework and its first measured pilot.</p> <p>Period: design from late June 2026; implementation and field hardening during July 2026; first pilot July to August 2026.</p> <p>Role: architecture owner and product owner, defining the control model, gates, and acceptance decisions.</p> <p>Contribution boundary: named LLM agents performed design work, implementation, scope checking, and review; the human approved gates, adjudicated findings, and retained final authority.</p> |

## 1. Executive summary

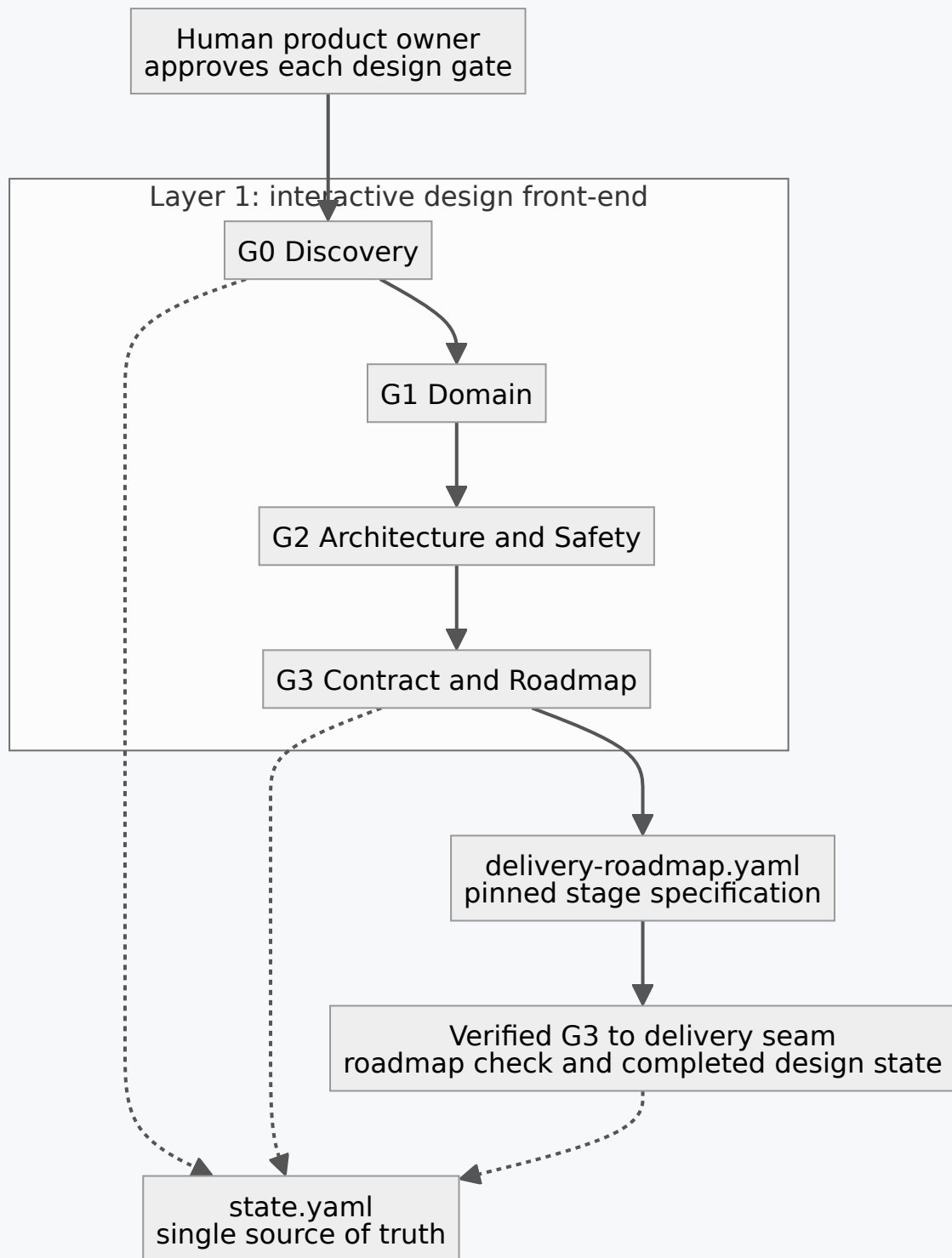
---

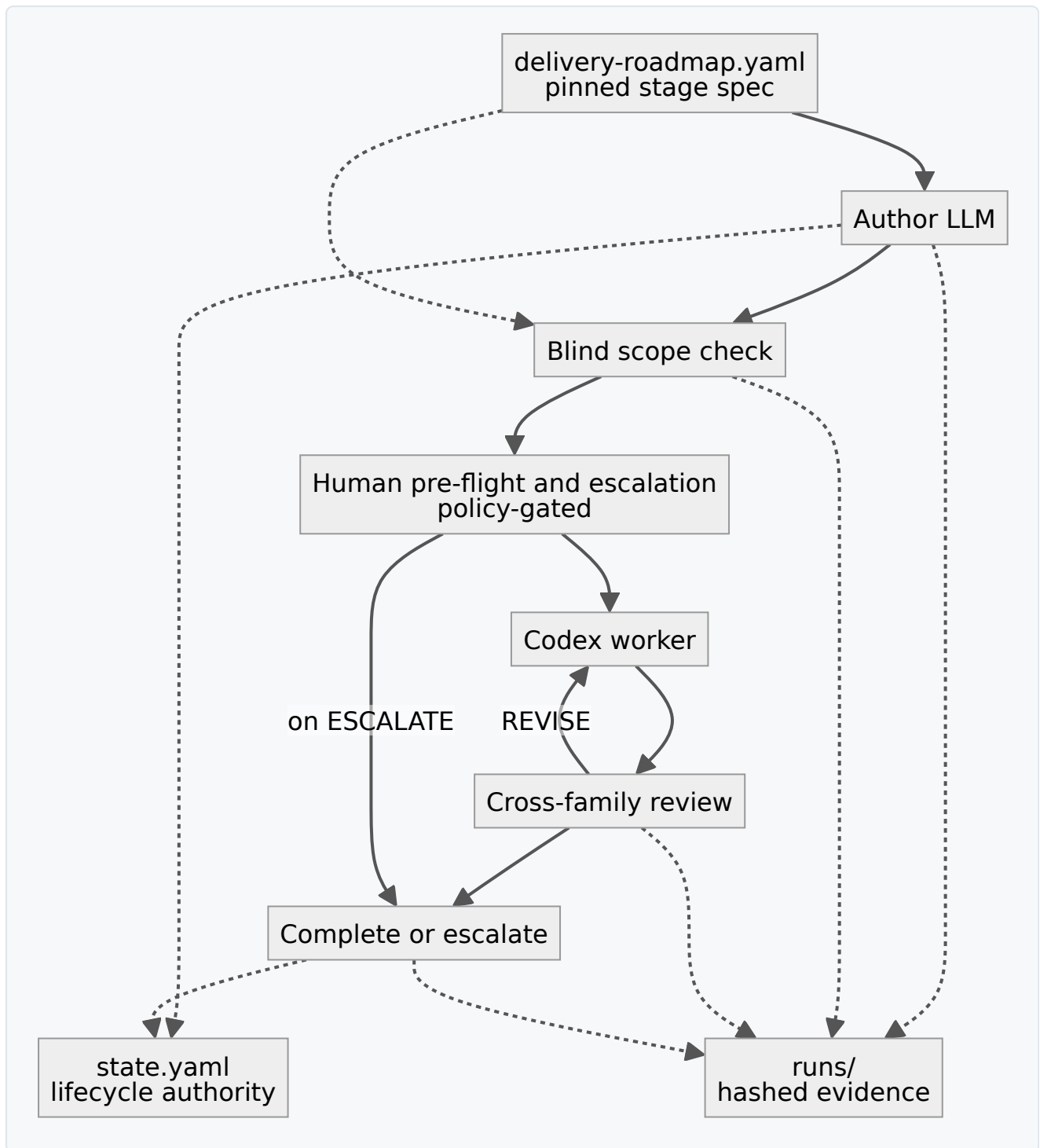
Factory\_FW is a reusable framework that turns a product idea into reviewed software through two connected layers: an interactive design process, where a human product owner approves Discovery, Domain, Architecture and Safety, and Contract and Roadmap gates, followed by a file-state delivery loop that authors each stage, checks its scope independently, sends it to a Codex implementation worker, reviews the result with a different model family, and records completion evidence. It differs from generic “AI writes code” workflows because code generation is only one controlled transition inside a larger system: the roadmap defines allowed work, `state.yaml` records what is authoritative, independent judges can stop dispatch, autonomy requires explicit human approval, and failures remain visible as resumable or escalated states. The framework uses LLM agents openly, while the human remains responsible for gate approval, risk acceptance, and escalation decisions.

## 2. Two-layer architecture as-built

---

Factory\_FW closes a gap between two different kinds of work. Product discovery and architecture need dialogue, revision, and explicit human decisions. Implementation can run for hours and must survive process exits, machine sleep, failed model calls, and repeated review. The shipped architecture keeps those modes separate and connects them through a verified seam.



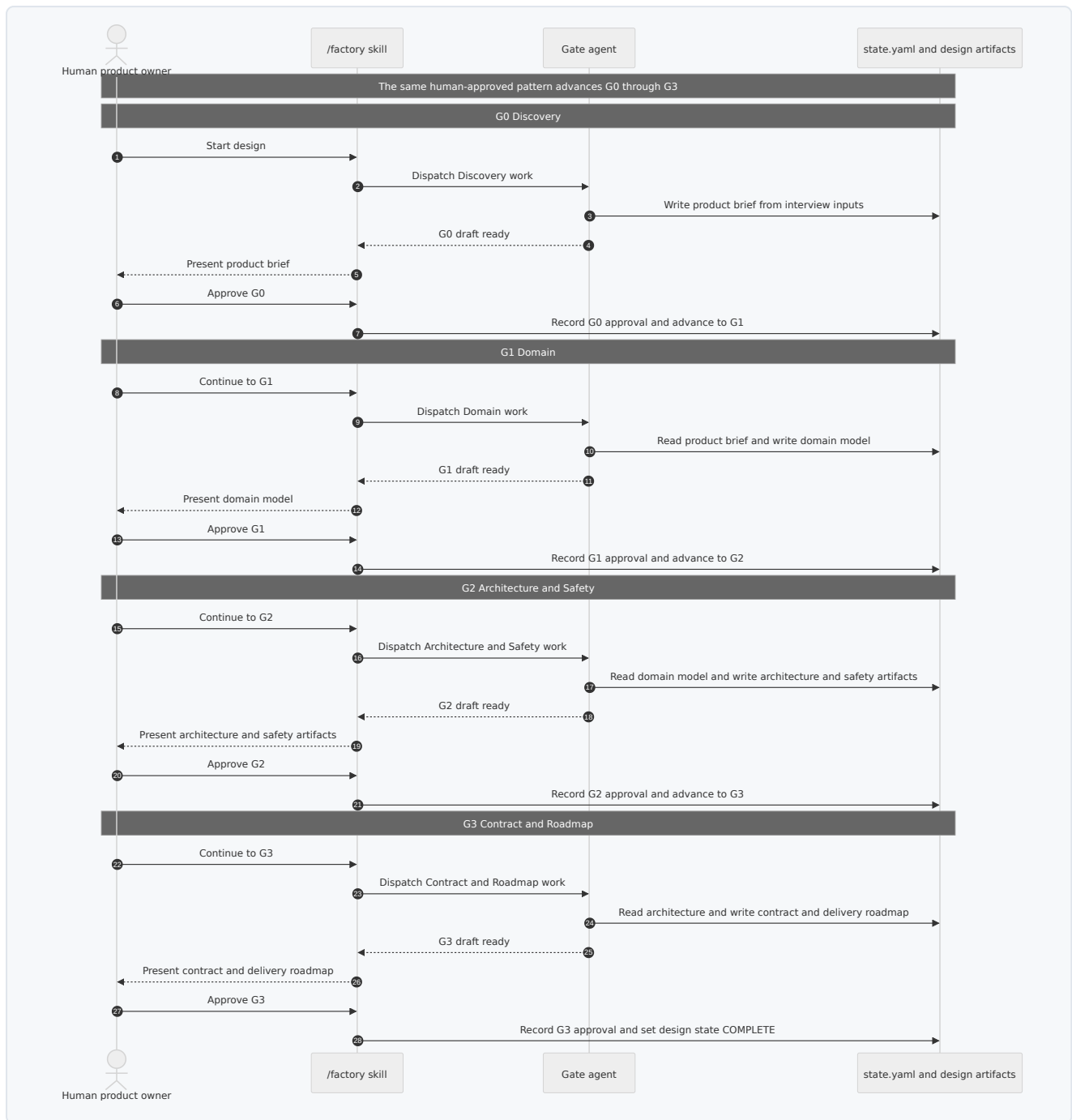


## Runtime view

The static views above show the two layers and their shared state. The following sequences show how human approval advances the design gates and how `fwctl` handles an author call at runtime.

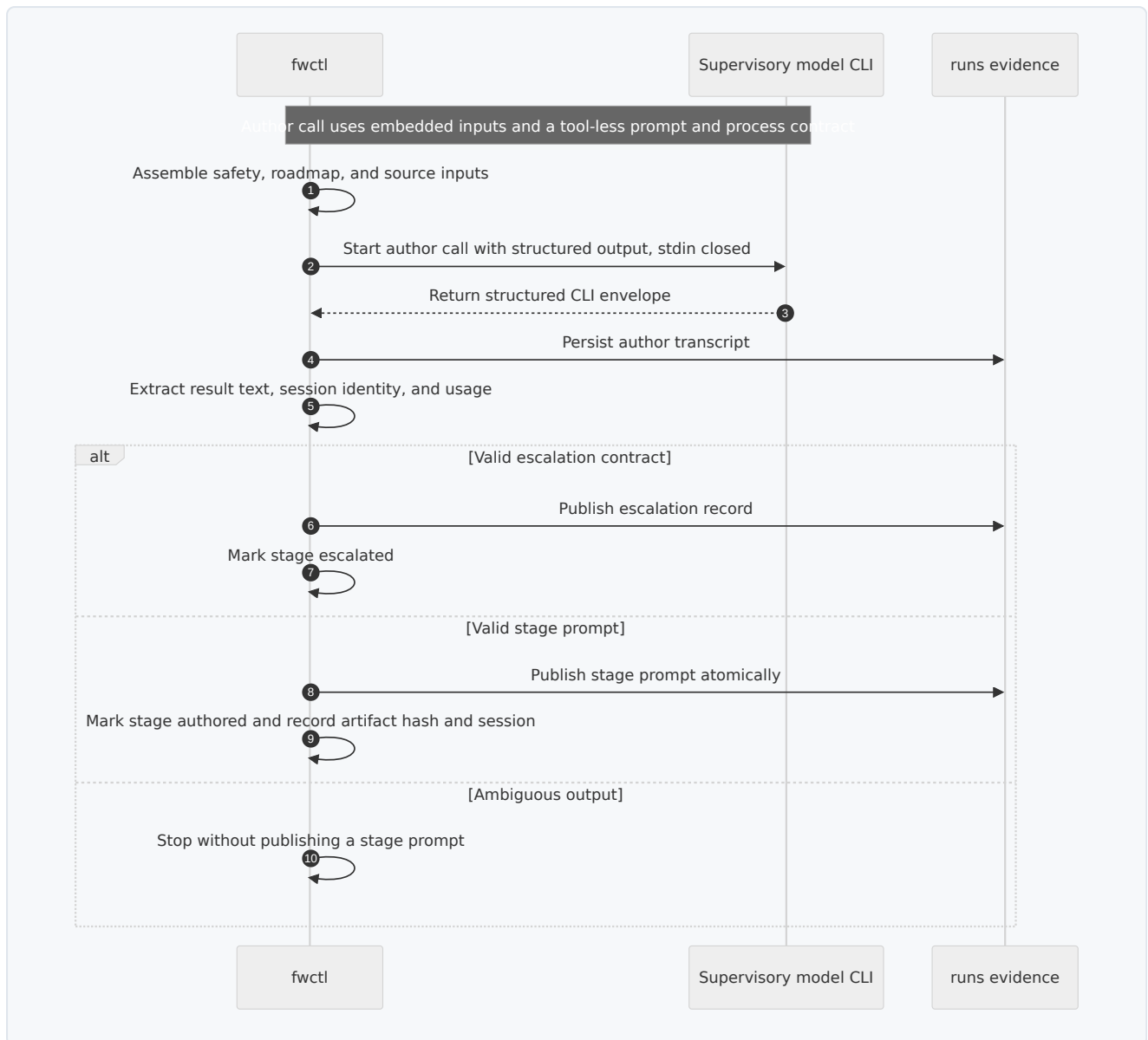
### Design-gate flow

One Factory invocation handles one gate. The skill dispatches a gate agent, the agent writes the next artifact from that gate's upstream inputs, and the human product owner reviews the result before the skill advances the state pointer.



## Author dispatch through `fwctl`

Supervisory author, scope-check, and review calls receive embedded inputs and operate under a tool-less prompt and process contract. `fwctl` performs their file reads and writes. The separate implementation worker deliberately receives workspace access because it implements the stage.



## Layer 1: design authority before implementation

The Layer 1 skill is invoked through the Factory command surface and presents four gates in a fixed order. The Gate Contract in the skill forbids skipping, merging, or silently reordering them. Ordering is a skill-level contract. For this gate-to-delivery transition, the only hard machine precondition is enforced at seam activation: design state must be marked **COMPLETE** before delivery can activate.

| Gate                       | Purpose                                                                            | Principal artifacts                                                                       | Human position                                  |
|----------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------|
| G0 Discovery               | Establish the problem, users, constraints, and delivery context.                   | Product brief and stack profile.                                                          | Interview, revise, then approve or reject.      |
| G1 Domain                  | Define the domain model and its invariants.                                        | Domain model.                                                                             | Review the model before architecture begins.    |
| G2 Architecture and Safety | Define structure, trust boundaries, data shape, and project-specific safety rules. | Architecture, storage contract, human safety brief, and machine-readable safety manifest. | Approve architecture and safety together.       |
| G3 Contract and Roadmap    | Define interfaces and decompose delivery into verifiable stages.                   | API contract plus human-readable and machine-readable delivery roadmaps.                  | Approve only after the roadmap verifier passes. |

The G3 handoff is a hard seam. `fwctl seam activate` requires design state to be complete, re-runs the roadmap completeness checks, pins the roadmap file hash, derives a specification hash for each stage, initializes the planned-stage records, and activates delivery through one guarded state mutation. Layer 1 stops there. It does not execute product code or advance delivery status itself.

## Layer 2: delivery as an explicit state machine

Layer 2 is a Node CLI named `fwctl`, distributed as one bundle. For each planned stage it coordinates the following as-built sequence:

Model assignments are overridable defaults: the scope check uses a Sonnet-class model by default, and post-implementation review uses an Opus-class model by default.

1. The author role reads the pinned roadmap entry and resolved design references, then drafts a stage prompt containing the universal boundary, the project safety manifest, and stage-specific restrictions.
2. A fresh Sonnet scope-check call receives only the roadmap stage and the generated prompt. A security-critical stage receives a second adversarial security lens. A failing result is preserved and dispatch stops.
3. Human pre-flight is required whenever the active policy says so. A fresh target begins with a blanket pre-flight requirement.
4. The Codex worker implements the stage on a stage-specific feature branch, runs the named validations, writes a report, and creates a commit with the required stage prefix.
5. `fwctl` verifies the report and independently searches git for a new matching implementation commit. A worker statement alone is not accepted as proof.
6. A fresh Opus reviewer receives the stage contract, prompt, worker report, safety material, and capped git evidence. It returns a schema-validated APPROVE, REVISE, or ESCALATE verdict.
7. Completion independently re-reads and hash-checks the review artifact, requires APPROVE, appends immutable completion evidence, and advances the stage pointer. REVISE resumes the existing Codex thread with a correction feed. ESCALATE stops for a human.

The author, scope checker, worker, and reviewer are separate invocations. Claude-family session identifiers are recorded for audit but are not reused between supervisory calls. Codex thread identity is reused only for the explicit resume path.

## **state.yaml : one authority, two owned sections**

`state.yaml` is the single source of truth for lifecycle state, not a narrative progress note. Layer 1 owns `design`; Layer 2 owns `delivery` after activation. The file records schema version, mutation generation, approved design gates, artifact paths, active roadmap hash, current stage, planned-stage states, budget counters, autonomy approval, completed-stage history, revisions, roadmap amendments, and loop halt information.

The roadmap and the state file have different authority. `delivery-roadmap.yaml` is the pinned specification for what each stage is permitted and required to do. `state.yaml` is the authoritative record of what the framework has done, what evidence exists, and what transition may happen next. Prompts and verdicts under `runs/` are content-hashed evidence referenced by state; they are not an alternative status database.

Every mutation goes through the same state API. A generation counter supports compare-and-swap checks, a lease lock prevents concurrent writers, and completed-stage and roadmap-amendment arrays are append-only. This design allows the interactive skill and the headless-capable loop to share state without maintaining competing trackers.

## **3. Safety and quality mechanisms**

---

Factory\_FW assumes that model output can be incomplete, structurally invalid, over-broad, or simply wrong. Its controls therefore sit around the implementation worker and around the shared state, rather than relying on one large prompt to behave perfectly.

## Mechanisms and the failures they contain

| Mechanism                 | As-built behavior                                                                                                                                                                                      | Failure contained                                                                                                            |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Roadmap completeness gate | Every stage needs testable acceptance criteria, at least one design citation, and validation commands. Security-critical stages also need an explicit tenant-boundary rule and negative confirmations. | An under-specified plan cannot activate delivery.                                                                            |
| Blind scope check         | A fresh Sonnet call compares the generated prompt with the pinned roadmap entry. It receives neither the author's private context nor other workspace content through fwctl's input assembly.          | Prompt omissions and scope additions are detected before code is written.                                                    |
| Security lens             | Security-critical stages receive a separate adversarial sanity lens and a security-aware review lens.                                                                                                  | A normal scope pass cannot silently substitute for an authorization or isolation check.                                      |
| Cross-family review       | Codex writes implementation; an Opus reviewer evaluates business and architecture coverage using the contract, report, safety material, and verified git evidence.                                     | The implementation model does not approve its own delivery result.                                                           |
| Autonomy lock             | Auto dispatch requires a complete recorded approval with approver, time, and rationale. New state also enables blanket human pre-flight.                                                               | A command-line switch alone cannot create authorized autonomy.                                                               |
| Crash-safe state          | Stage status, sessions, hashes, and timestamps are persisted through atomic writes under a lease lock. Timeouts and missing commit proof become explicit stalled states.                               | A killed process does not require reconstructing progress from chat history.                                                 |
| Git and artifact proof    | Dispatch refuses unrelated worktree changes, refuses to operate on protected branches, snapshots framework-owned state, and verifies a new stage commit independently.                                 | A report cannot impersonate a commit, and framework snapshots do not sweep unrelated product changes into their own commits. |
| Structured verdicts       | Scope and review answers must match schemas. Malformed output is preserved for diagnosis and does not become a passing verdict.                                                                        | Free-form model prose cannot be mistaken for a valid control decision.                                                       |
| Append-only completion    | Completion history cannot shrink or be edited. Corrections use a revision record or a new corrective stage.                                                                                            | A later retry cannot rewrite the record of what was previously approved.                                                     |
| Bounded loop              | Budget checks, call timeouts, a finite iteration cap, repeated-REVISE escalation, and halt notifications bound unattended execution.                                                                   | Non-converging or unexpectedly costly runs stop with a recorded reason.                                                      |

The autonomy rules are deliberately layered. With the shipped seed, every stage waits for human pre-flight. A human may later relax that blanket rule and record an autonomy approval. Even then, security-critical stages remain human-gated. The first two planned stages remain gated until the audited autonomy object is complete. Every ESCALATE outcome requires a human regardless of mode.



Crash-safe resume is not merely “run the command again.” The framework distinguishes a live dispatch from a stalled one, records the Codex thread identity, verifies that stored artifacts still match their hashes, and routes recovery to `dispatch --resume`. When a reviewer requests changes, the resumed worker receives a structured correction feed. When a prior worker wrote a report but failed commit verification, the resume prompt names that failure and requires commit proof before reporting again.

## July 2026 hardening: failures were retained as design input

The initial v1 build was followed by field-driven corrections rather than a claim that the first design was final.

- One early hardening pass corrected assumptions exposed by real Claude and Codex CLI runs. The implementation learned the actual result-envelope shapes, closed worker stdin, captured the correct resume identity, tolerated fenced verdict JSON, accounted for usage on failure paths, required a worker commit, and supplied git evidence to review.
- A subsequent hardening change aligned push behavior and exit-code semantics after a cross-command consistency sweep.
- A later fix repaired an operational dead end. Early v1 documentation admitted that an activated roadmap could not be replaced. The current implementation can verify and regenerate the pinned roadmap, mark changed unfinished stages stale, add corrective stages, and still refuse changes to completed-stage specifications.
- A redesign followed a safety-guard failure. Semantic review paths had been compiled into mutation blocks, which blocked legitimate source edits without enforcing the semantic rule itself. The redesign made those paths review metadata, introduced an explicit hard-block list for direct file-tool targets, generated a human review checklist, and narrowed the shell guard to documented deny-biased checks. Subsequent adversarial review found option, exclusion, abbreviation, glob, and path-specification bypasses; each became a regression case.
- The final hardening commit before this edition followed a pilot failure in which a successful author response was misclassified as an escalation because harmless prose mentioned escalation markers. The detector contract was tightened, ambiguous author output became fail-closed, and no artifact is published when output is neither a valid stage prompt nor a valid escalation. The same commit made vendoring refuse dirty deployable source paths unless the operator explicitly accepts and records that provenance.

## Current limitations

The controls have defined boundaries and should not be described as complete assurance.

- The generated local hook is a best-effort backstop. It checks direct file-tool targets and a narrow set of destructive shell patterns; it does not enforce semantic safety rules. Review, tests, continuous integration, and branch protection remain necessary.
- Supervisory `claude -p` calls are assembled from embedded inputs and are invoked without tool-access arguments. The implementation reference describes tool-less operation as a prompt and process contract, not as an independent operating-system sandbox. The supervisory roles need no write access because `fwctl` performs all file reads and writes itself; the Codex dispatch call is the opposite case by design.
- `fwctl` creates stage branches, verifies commits, and can push only when explicitly enabled. It does not create or update pull requests; that integration remains outside the current CLI.

- Roadmap dependencies are validated structurally but do not control ordering. Array order and the current-stage pointer do.
- The per-stage specification hash is compared during roadmap regeneration, not by every ordinary stage command. Whole-roadmap drift is checked broadly.
- Codex dispatch token use is recorded, but its dollar cost is not included in the framework's USD budget counter.
- Target `runs/` retention is not installed by the vendoring script, so long-running projects need their own retention policy.
- Status: deferred, not implemented in v1. The parallel in-session security Workflow and an independent checker for the G0 through G3 design gates remain outside the shipped version.

## 4. From design to implementation

---

In this as-built genre, the R1 through R10 disposition table and the principal diagram-delta table together serve as the architecture decision record: they capture the decision pressure, shipped outcome, and reason for change without recasting the history as ADR-XXX entries.

The pre-implementation architecture diagrams are valuable design history, but their own header says “DESIGN” and “pre-implementation.” They establish intent, not runtime truth. The two-layer boundary, four design gates, blind scope check, human authority, and vendored deployment model survived. Several mechanisms changed before or after implementation.

## Principal diagram deltas

| Pre-implementation representation                               | As-built reality                                                                                                                                      | Reason for the change                                                                                                                                   |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| A Bash dispatcher named <code>orchestrate.sh</code> .           | A dependency-light Node CLI bundled as <code>fwctl.mjs</code> .                                                                                       | The second review concluded that state, schema, process, and test behavior would be easier to prove in one program than in shell plus external parsers. |
| A simple directory mutex and Boolean autonomy flag.             | A heartbeat lease lock, holder-side loss detection, atomic writes, generation CAS, and an audited autonomy object.                                    | Long-running and concurrent execution needed stronger crash and mutation semantics.                                                                     |
| Delivery state shown under the payload directory.               | The payload is under <code>.factory-fw/</code> ; authoritative <code>state.yaml</code> and <code>runs/</code> stay at the target workspace root.      | Re-vendoring must replace framework code without replacing project delivery history.                                                                    |
| Completion shown opening or updating a pull request.            | The CLI enforces stage branches, commit evidence, protected-branch refusal, and opt-in push. Pull-request creation is external.                       | Only behavior backed by current source is claimed here.                                                                                                 |
| Permission profiles described as the supervisory-call boundary. | Supervisory calls use embedded inputs and no tool-access arguments; the remaining tool-less guarantee is documented as a prompt and process contract. | The shipped call shape differs from the design proposal and must not be overstated.                                                                     |
| Semantic safety paths compiled into runtime blocks.             | Semantic paths feed review; a separate hard-block list drives direct mutation denial.                                                                 | Field use showed that syntax-level path blocking could prevent valid work without proving the semantic property.                                        |

## R1 through R10: review findings and shipped dispositions

| Review | Finding                                                                                             | Shipped disposition                                                                                                                                                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1     | The roadmap was too thin for a blind checker to detect consequential omissions.                     | Accepted. The roadmap gained acceptance criteria, source citations, owned modules, data contracts, validation commands, rollback, negative confirmations, and correction feed, plus a semantic completeness gate.                       |
| R2     | One state file lacked a safe concurrency and migration contract.                                    | Accepted with later refinement. Lease locking, atomic writes, CAS, schema checks, FSM checks, and invariants shipped. Automatic migration did not; v1 is greenfield, and re-vendoring refuses schema mismatch unless explicitly forced. |
| R3     | “Approve once, then run automatically” was weaker than the donor process.                           | Accepted with modification and human confirmation. Pre-flight became mandatory for security-critical work, early autonomy became earned, and the shipped seed is more cautious by gating every stage until relaxed.                     |
| R4     | The proposed parallel security Workflow could not operate through the chosen process boundary.      | Accepted with modification and human confirmation. v1 uses sequential security lenses in sanity and review plus mandatory human pre-flight. Status: deferred, not implemented in v1, for the parallel mechanism.                        |
| R5     | A generic safety statement lacked project-specific force.                                           | Accepted. G2 produces human and machine-readable safety manifests, required tests, negative confirmations, review checks, and emitted local controls. Later hardening separated semantic review scope from direct mutation blocks.      |
| R6     | Reimplementing the donor loop could silently omit safety behavior.                                  | Accepted. The port began with 23 golden detect decisions, then added command-level and real-CLI validation.                                                                                                                             |
| R7     | Automatic git push behavior was underspecified.                                                     | Accepted. Stage branches, dirty-worktree refusal, protected-branch refusal, explicit commit verification, and opt-in push shipped. Automated pull-request creation did not.                                                             |
| R8     | Reopening a completed stage would contradict append-only audit history.                             | Accepted. Completed records are immutable; correction uses an appended revision record or a new stage introduced by roadmap regeneration.                                                                                               |
| R9     | Model routing was written as policy but not verified at startup.                                    | Accepted. Preflight checks the model binaries, role configuration, timeouts, state, budget shape, and active-roadmap integrity before the loop proceeds.                                                                                |
| R10    | Original Factory rules said there were no external process calls, contradicting the unified design. | Accepted. The current skill explicitly documents <code>fwctl</code> , Claude, Codex, and git process boundaries while keeping state mutation behind the common CLI.                                                                     |

A second pre-implementation review added mechanisms beyond R1 through R10: the Node substrate, structured model verdicts, usage capture, a full per-stage FSM, lease locking, roadmap pinning, an audited autonomy object, and machine-emitted safety artifacts. One proposed item was superseded rather than copied literally: the permission-profile design became the narrower call shape described above. One operational item shipped only after the first as-built documentation exposed the gap: roadmap regeneration and stale-stage production.

This history matters because the framework's design record is not presented as prophecy. The durable architectural decisions are the separation of design and delivery, independent verification,

human authority, one state API, and evidence-based transitions. Substrate choices and enforcement mechanics were allowed to change when tests, live process behavior, or pilot use contradicted the earlier diagrams.

## 5. Validation

Validation has distinct evidence layers: framework-side tests and real process exercises, then a measured pilot in a separate product repository. They answer different questions and are kept separate here.

### Framework-side evidence

Framework validation appears in three successive repository-recorded snapshots. The v1 completion record reports 126/126 tests and three end-to-end rounds against the real Claude and Codex CLIs. The final round completed author, sanity, real Codex dispatch with independently verified feature commit, review APPROVE, completion, and the all-stages-completed exit, followed by pause, budget, and roadmap-drift drills. A later snapshot records 152 tests, 152 passed, 0 failed. The subsequent hardening snapshot records 155 pass, 0 fail after adding author-output and deployment-provenance regressions.

These snapshots were not rerun for this public edition, and they must not be added together. The latest recorded result does not erase the earlier sequence; it shows how the suite changed as field defects became regression cases.

### Pilot evidence (author's own multi-tenant SaaS project)

The first pilot applied the two-layer process to the author's own multi-tenant SaaS project. The following figures are copied from the portfolio assignment without recomputation:

| Pilot evidence                                           | What it supports                                                                                        |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 7 stages                                                 | The roadmap was executed as a multi-stage delivery, not a single generation request.                    |
| 93 commits                                               | Delivery left a granular repository history.                                                            |
| 22 merged PRs                                            | Integration decisions were recorded through reviewed change units.                                      |
| 14 fix or amendment branches                             | Correction work remained visible instead of being hidden inside one final snapshot.                     |
| 19 075 LOC application plus 12 350 LOC tests in 45 files | The run covered an application and a substantial test body across multiple files.                       |
| about 5 weeks                                            | The framework operated across a multi-week delivery period.                                             |
| autonomy lock                                            | Unattended progression remained subject to an explicit framework control.                               |
| human input limited to gate approvals                    | The human retained decision authority without manually writing the application during the measured run. |

The pilot figures show scale, duration, review activity, and visible self-correction. They do not prove that every change was correct, that code volume is quality, or that one result generalizes to every product. The 14 fix or amendment branches are useful precisely because they contradict a zero-defect story: the system found or received corrections and carried them through auditable change

units. The July author-output failure also demonstrates that the pilot tested the framework itself, not only the product; it produced a concrete fail-closed hardening commit.

For the product context, stage summary, and concise evidence narrative, see the companion *Case sheet: Agentic delivery pilot — multi-tenant SaaS (v1, po Opus review)*. This as-built reference explains the reusable control system; the case sheet explains the pilot outcome.

## 6. Deployment model

In `Factory_FW`, “deployment” means vendoring the framework into a target repository. It does not mean deploying the target application or changing cloud infrastructure. The editable master retains source, design records, tests, and packaging tools; a target receives only the payload needed to design and deliver that target.

### Vendored payload

`tools/deploy-into.sh` installs the coordinated payload parts:

| Master payload                                    | Target location                        | Responsibility                                                         |
|---------------------------------------------------|----------------------------------------|------------------------------------------------------------------------|
| <code>framework/skill/factory</code>              | <code>.claude/skills/factory</code>    | Layer 1 gate instructions, agent roles, templates, and stack profiles. |
| <code>framework/orchestrator/bin/fwctl.mjs</code> | <code>.factory-fw/bin/fwctl.mjs</code> | Layer 2 executable bundle.                                             |
| <code>framework/config</code>                     | <code>.factory-fw/config</code>        | State seed, universal boundary, and safety-manifest starter.           |

The script also writes `.factory-fw/VERSION` with the exact `Factory_FW` commit and deployment time. At current HEAD it checks whether the deployable source paths are dirty. A normal run refuses dirty provenance. An explicit override is possible, but the version stamp then records that the payload did not come from a clean tracked state.

Project state stays outside the replaceable payload. Re-vendoring replaces the skill, bundle, and configuration while leaving root `state.yaml` and `runs/` untouched. This is what permits deliberate framework updates during a project without silently resetting its delivery history. A schema-version mismatch is refused unless the operator explicitly forces the operation; the script does not attempt automatic state migration.

Only `framework/` is copied. Design documentation, source tests, golden donor material, and packaging tools stay in the master. The target therefore runs a pinned bundle rather than importing a mutable global installation. Updates occur when a human deliberately re-runs the vendoring script; they do not arrive silently in the middle of a build.

### Relationship to UDF2

UDF2 is a read-only donor, not a runtime component. `Factory_FW` studied and reimplemented selected delivery-loop patterns: the author, sanity, dispatch, review, and complete cadence; independent checking; correction and escalation; staged artifacts; and durable progress. The donor repository was not modified by the `Factory_FW` build, and a target does not need UDF2 installed to run the vendored payload.

A provenance copy of donor material exists under `tests/golden/donor/` so the reimplementation can characterize retained, changed, and dropped behavior. That copy is a development and test reference only. It is not part of the vendored target. This boundary matters both operationally and ethically: the new framework can evolve its substrate and safety controls without rewriting a live donor or pretending that copied patterns are a shared runtime dependency.

The deployment model therefore separates authority: the `Factory_FW` master owns framework source; the version stamp identifies the vendored revision; target `state.yaml` owns project lifecycle state; and the human decides when a new framework revision may cross into an active target.

## 7. Lessons Learned

---

- **Treat the roadmap as an executable contract.** A stage description becomes governable only when it includes acceptance criteria, design citations, owned modules, validation commands, rollback, and explicit negative confirmations. The R1 and R5 dispositions turned those elements into activation checks rather than editorial guidance.
- **One state authority needs concurrency and history rules together.** Lease locking, atomic writes, generation checks, immutable completion records, and append-only corrections address different parts of the same problem: a resumable workflow must not create competing versions of what happened. R2 and R8 made that boundary explicit.
- **Autonomy is an earned state, not a command-line shortcut.** The R3 and R4 changes retained human pre-flight for security-critical work, made early autonomy subject to recorded approval, and kept every escalation human-owned.
- **Model integration must be tested against real process behavior.** July hardening corrected result-envelope assumptions, stdin handling, resume identity, fenced verdict parsing, failure-path usage capture, commit proof, and exit semantics. A plausible prompt contract was not enough.
- **Semantic review and direct mutation denial are separate controls.** Treating review paths as file blocks prevented legitimate work without proving the safety property. The redesign separated review metadata from the narrow hard-block list and converted discovered bypasses into regression cases.
- **Ambiguous evidence must stop publication.** The author-output failure showed that keyword matching can misclassify successful work. Structured contracts, independent commit checks, schema-validated verdicts, and fail-closed handling now prevent prose or a report from impersonating completion.
- **As-built documentation must preserve design change, not hide it.** The R1 through R10 dispositions and later diagram deltas distinguish retained principles from superseded mechanisms, including the Node substrate, process boundaries, roadmap regeneration, and external pull-request handling.